

Offloading Visual-Inertial Odometry for Low Power Extended Reality

Qinjun Jiang* Muhammad Huzaifa William Sentosa Jeffrey Zhang Steven Gao Yihan Pang
Brighten Godfrey Sarita Adve
University of Illinois Urbana-Champaign, USA

ABSTRACT

Power consumption is a constraint in designing all-day wear extended reality (XR) devices. To reduce power consumption, we present a novel end-to-end XR system with visual-inertial odometry (VIO) offloaded to a remote server. Even with long network latency, the system produces an end-to-end XR experience comparable to a system without offloading while providing significant power savings. This latency and bandwidth-resilient nature of head tracking is a new and surprising result.

Index Terms: Human-centered computing—Virtual reality—Human-centered computing—Mixed/augmented reality

1 INTRODUCTION

XR has shown the potential of transforming many aspects of human life. However, there is currently an orders of magnitude gap between the power, compute capacity, and quality of experience of current and desired XR devices [6]. Bridging this gap requires many advances to improve the power consumption of individual components of an XR system while providing a high-quality end-to-end user experience. This work presents a way of reducing the power consumption from head tracking, one of the top compute- and power-intensive components [6] in a tetherless XR device, by offloading it to a remote server.

At first glance, it seems counter-intuitive to offload head tracking. This is a critical component that follows the user's head motion to produce the user's 6DOF pose, which is used in many parts of the XR workflow, including rendering the new frame from the new perspective of the user. It is highly latency sensitive in the sense that if the pose was not accurate enough or arrived late, the user would see the display from the wrong perspective. The long and unpredictable network latencies over wireless networks can potentially make the user experience with offloaded head tracking unacceptable.

We make the following critical observations to mitigate the impact of network latency. State-of-the-art head tracking uses visual-inertial odometry (VIO) algorithms that rely on cameras and inertial measurement units (IMUs). The VIO is computer vision based and operates on the camera frame rate (tens of frames per second). It produces accurate poses but does not keep up with the pose query frequency in an XR system (e.g., rendering at 120Hz). To compensate the long latency between two accurate VIO poses, XR systems can use IMU integration to generate less-accurate poses at the IMU frequency of hundreds of hertz by adding a relative pose integrated from IMU samples to the latest received VIO pose. However, because IMUs are noisy, the infrequent VIO poses are still needed to correct the accumulated error and provide proper IMU biases to arrest IMU drift.

Most XR systems include another component called pose prediction to answer pose queries for future timestamps. Assuming humans retain their momentum in a short time span, the pose prediction generates a predicted pose based on the user's current velocity or acceleration. Since the quality of the predicted poses reflects the

*e-mail: {qinjun2, huzaifa2, sentosa2, jfzhang2, hongyig3, yihanp2, pbg, sadve}@illinois.edu

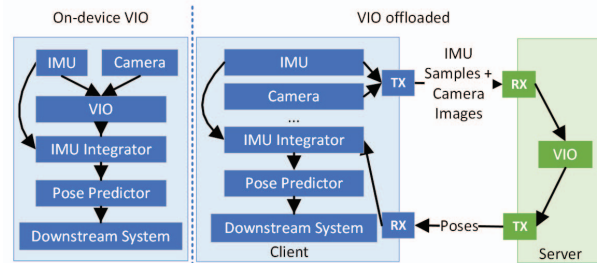


Figure 1: System without (left) and with (right) VIO offloaded.

end-to-end tracking quality and directs what the user experiences, we compare that instead of the VIO poses against the ground truth poses across different settings for measuring the quality of the system.

We also evaluate the power savings of offloading VIO by comparing the power consumption of our offloading system with the full system on an embedded class platform.

Our results show that the system with VIO offloaded produces tracking qualities that are similar to a fully on-device system. The result is verified by inspecting the rendered frames through a VR headset in real-time. The system is robust to hundreds of milliseconds of network latency with less than 8Mbps bandwidth utilization and produces a net device power reduction of 22%.

We summarize the key system aspects and results below and refer to the full paper for details.¹

2 SYSTEM

Our system builds upon ILLIXR, a state-of-the-art open-source end-to-end XR system and research testbed [6]. ILLIXR is a highly modular system composed of a set of plugins. We employ the client-server model in our offloading setup and divide the original ILLIXR system into two groups: the device-side components and the server-side components. The VIO exists on the server-side while the other components remain on the device-side. Leveraging the modular features and the central communication framework of ILLIXR, we develop plugins to handle the transmission and reception on both the device-side (device_tx, device_rx) and the server-side (server_tx, server_rx). The original system and the offloaded system are illustrated in Figure 1. The client and server communicate via TCP sockets using Linux's default settings. Specifically, the client establishes two connections to the server: one for sending IMU-camera frames and the other for receiving poses.

Transmitting raw camera images places stringent requirements on bandwidth availability. For instance, the EuRoC dataset [1] we use would require around 120 Mbps when transmitted without compression. This bandwidth requirement is either beyond the availability of many commonly used consumer networks and/or incurs a high cost and interferes with other traffic. Therefore, we perform H.264 compression on the raw camera frames using GStreamer gst-v4l2 codec, which is hardware accelerated by NVIDIA NVENC. We control the compression ratio by adjusting the compression bitrate. We

¹The extended version can be found at <https://tinyurl.com/4xmfc2ccc>. Dhakal et al. [2] concurrently propose offloading major computations of simultaneous localization and mapping (SLAM) using a similar approach as ours. But we do a more comprehensive evaluation of effects of network latency, bandwidth, and compression on an end-to-end system.

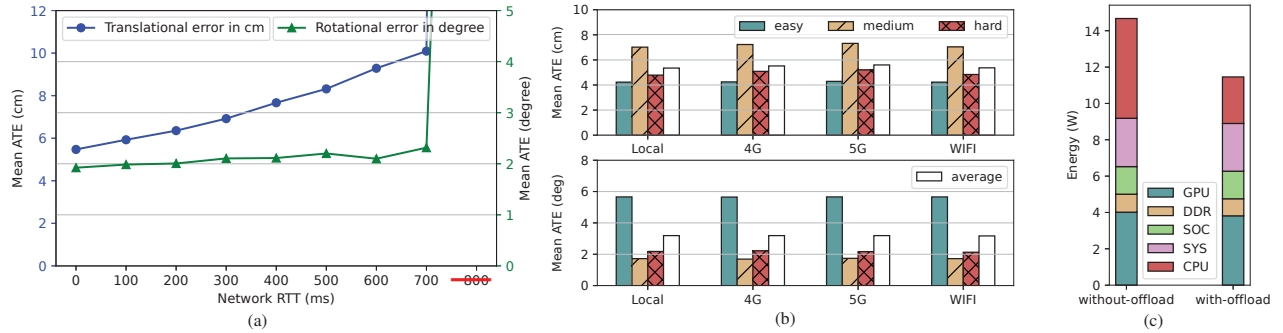


Figure 2: Experiment results: (a) Absolute trajectory (translational and rotational) error with different network round trip times. Tracking fails when the RTT is 800ms, indicated by the red bar. (b) Absolute trajectory (translational and rotational) error when the device is connected to 4G, 5G, and WiFi. We also report the error for localhost (local). (c) The power decomposition on Jetson.

use the default values in `gst-v4l2` for other configurations (Constant Bitrate, `iframinterval` of 30, Baseline profile).

We use OpenVINS [3], an open-source state-of-the-art VIO implementation, as our VIO component. For the IMU integrator, we use GTSAM [5] and apply a low-pass filter to smooth out jitters introduced by network latency. We expect the results to be extendable to other implementations of VIO and IMU integrators.

3 EVALUATION

We conduct the experiments on a high-end desktop PC with Intel Core i9-10900K CPU, NVIDIA GeForce RTX 3090 GPU, and Ubuntu 20.04.

Datasets, Metrics, and Applications. We use the three Vicon Room 1 datasets from the EuRoC MAV Dataset [1] which represent different levels of tracking difficulty. To evaluate the tracking quality, we calculate the absolute trajectory error (ATE) by comparing the predicted poses with the ground truth using the `evo` package [4]. We use a lightweight OpenGL-based VR application unless mentioned otherwise. Nevertheless, our system is application-agnostic.

Latency impact. To investigate the network latency impact on the tracking, we run both client and server on the same machine via localhost and use Linux Traffic Control utility to emulate the network latency. We set the targeted compression bitrate at 5Mbps. Figure 2a shows that the Mean ATE steadily increases as the network latency increases, but the tracking fails when the network RTT is 800ms. This can attribute to the noisy nature of IMU. Integrating over too long a period without getting corrections from accurate VIO pose will lead to drifting. This is also the reason that we can not fully rely on IMU.

Deployment on the cloud server. We demonstrate the viability of deploying the server to the nearest AWS EC2 instance located in Ohio. We use the desktop PC to run the client part of the system. We then connect the client through three wireless interfaces: WiFi, 4G, and 5G. To further reduce network congestion, we use a compression bitrate of 0.5Mbps.² Figure 2b shows the mean ATE when the client is connected to different network interfaces. We also compare them with localhost (with zero latency) where both client and server run on the desktop PC. We find that experiments conducted on all wireless interfaces give comparable ATE to localhost as our system can tolerate high network latency. Specifically, the mean translational ATE of Vicon Room1 Medium on 4G is only 3% worse than localhost.

A complete user study of the visual XR experience with VIO offloaded is outside the scope of this paper. Nevertheless, to assess whether the XR experience corroborates the positive ATE observa-

²We test the system with different compression bitrates. The mean translational ATE of Vicon Room1 Medium increases by only 3.3 cm when using a bitrate of 0.5Mbps (leading to a 640× compression ratio) compared to no compression.

tions with the EuRoC dataset, one of the authors wears our Valve Index headset and has live experience with two real XR applications (Sponza and ARDemo). A ZED mini camera attached to the front of the headset feeds the system with IMU samples at 800 Hz and stereo images (of 672×376 pixels) at 30 fps as the user walks around in physical space. The user sees frames displayed by ILLIXR through the headset in real-time. Not knowing which experience is which, the user rates the experience on the offloaded system to be smooth and indistinguishable from the localhost.

Power consumption. We quantify how offloading VIO saves power by running ILLIXR on NVIDIA Jetson AGX Xavier development board and compare the energy consumption with and without VIO offloading. To emulate the limited power of typical VR devices, the Jetson runs at the 10 Watt preset with a 2.2 GHz CPU frequency, 1.3 GHz GPU frequency, and DVFS disabled. We then monitor the power rails on the Jetson to collect the power of CPU, GPU, DDR (DRAM), SoC (on-chip microcontrollers), and Sys (display, storage, and I/O). We connect Jetson with WiFi and leave 4G/5G study for future works. Figure 2c shows a decomposition of the power consumption for each component. With offloading, the total power consumption is reduced by 22%. While the power of other components is almost identical with or without offloading, the CPU power falls by almost 60% with offloading.

4 CONCLUSION AND FUTURE WORK

We have presented an end-to-end XR system with VIO offloaded which is surprisingly tolerant to high network latency and saves significant power on device. This work demonstrates the potential of using offloading to reduce the compute and power of an XR device and illustrates the importance of studying a complete end-to-end XR system.

ACKNOWLEDGMENTS

This work was supported in part by the Applications Driving Architectures (ADA) Center, a JUMP Center co-sponsored by SRC and DARPA, the DARPA DSSOC program, the National Science Foundation under grants 2120464 and 2217144, and by a gift from Cisco Systems Inc.

REFERENCES

- [1] M. Burri et al. The EuRoC micro aerial vehicle datasets. *IJRR*, 2016.
- [2] A. Dhakal et al. SLAM-Share: Visual Simultaneous Localization and Mapping for Real-Time Multi-User Augmented Reality. *CoNEXT*, 2022.
- [3] P. Geneva et al. OpenVINS: A Research Platform for Visual-Inertial Estimation. *IEEE ICRA*, 2020.
- [4] M. Grupp. `evo`: Python package for the evaluation of odometry and SLAM. <https://github.com/MichaelGrupp/evo>, 2017.
- [5] GTSAM repository. <https://github.com/borglab/gtsam>, 2020.
- [6] M. Huzaifa et al. ILLIXR: An Open Testbed to Enable Extended Reality Systems Research. *IEEE Micro*, 2022.